



Ministerul Educației și Cercetării
Universitatea "OVIDIUS" Constanța
Facultatea de Matematică și Informatică
Specializarea Informatică

Aplicatie software de pontare a participanților la evenimente

Lucrare de licență

Coordonator științific:

Conf. univ. dr. Puchianu Crenguța-Mădălina

Absolvent:
Moraru Vlad

Constanța
2026

Cuprins

Cuprins	i
Lista Figurilor	iv
1 Introducere	1
1.1 Structura lucrării	2
2 Definirea problemei	3
2.1 Introducere și Contextualizare	3
2.2 Analiza soluțiilor existente	4
2.2.1 Sistemele cu cartele magnetice	4
2.2.2 Sisteme mobile bazate pe scanare de coduri QR	4
2.2.3 Sisteme de tracking pe bază de localizare GPS	5
2.2.4 Sinteza comparativa	6
2.2.5 Fundamentarea arhitecturală a soluției PresSync	6
2.3 Descrierea problemei și soluția propusă	7
2.4 Obiectivele sistemului	8

2.5	Actori principali și delimitarea rolurilor	8
2.6	Scenarii de baza	9
3	Analiza Software a Sistemului	10
3.1	Documentul de cerințe	10
3.1.1	Actori și roluri în sistem	11
3.1.2	Cerințe funcționale	12
3.1.3	Cerințe nefuncționale	13
3.2	Cazuri de utilizare	14
3.2.1	Creare Cont	14
3.2.2	Vizualizare Prezență cu Statistică	15
3.2.3	Înregistrare Prezență	16
3.2.4	Administrează Prezență Manual	16
3.2.5	Creare Categori de Evenimente	17
3.2.6	Creare Evenimente	17
3.3	Diagrame de secvențe de sistem	18
3.3.1	Evenimente de sistem	18
3.3.2	Diagrame de secvență de sistem	20
3.4	Modelul domeniului	25
4	Proiectarea si implementarea aplicatiei	27
4.1	Arhitecturi si stilur arhitecturale	27
4.2	Modele de proiectare de atribuire a responsabilitatilor	28
4.3	Diagrame de interactiune	30
4.3.1	Creare Cont	30

4.3.2	Creare Evenimente	36
4.3.3	Inregistrare Prezenta	42
4.3.4	Vizualizare prezenta cu statistici	43

Lista Figurilor

3.1	Tipuri de relații în diagrama de cazuri de utilizare	18
3.2	Diagrama cazurilor de utilizare	19
3.3	Prezentarea diagramelor de secvență de sistem	21
3.4	Diagrama de secvență — Creare cont	21
3.5	Diagrama de secvență — Creare evenimente	22
3.6	Diagrama de secvență — Înregistrare prezentă	22
3.7	Diagrama de secvență — Vizualizare prezentă	23
3.8	Diagrama de context	24
3.9	Explicarea relațiilor în modelul domeniului	25
3.10	Diagrama de clase a modelului domeniului	26
4.1	Diagrama de interacțiune — Creare cont	30
4.2	Implementare AuthenticationController	30
4.3	Implementare AuthenticationService	31
4.4	Interfața de înregistrare — formularul gol	32
4.5	Interfața de înregistrare — eroare nume invalid	33

4.6	Interfața de înregistrare — eroare email invalid	34
4.7	Interfața de înregistrare — eroare parolă prea scurtă	35
4.8	Diagrama de interacțiune — Creare evenimente	36
4.9	Implementare EventCategoryController	36
4.10	Implementare CreateEventCategoryCommand	37
4.11	Interfața de creare categorii — butonul de creare	38
4.12	Interfața de creare categorii — panoul de creare	38
4.13	Interfața de creare categorii — eroare 1	39
4.14	Interfața de creare categorii — eroare 2	40
4.15	Interfața de creare categorii — eroare 3	41
4.16	Diagrama de interacțiune — Înregistrare prezență	42
4.17	Diagrama de interacțiune — Vizualizare prezență cu statistici	43

Capitolul 1

Introducere

Prezenței participanților la activități educaționale, profesionale sau organizaționale este o provocare în contextul actual. Metodele actuale de pontaj — apelul nominal, listele pe hârtie sau cartelele de acces fizice — sunt lente, predispuse la erori și vulnerabile, precum înregistrarea prezenței în locul unui coleg absent. Soluțiile digitale existente, bazate pe coduri QR, localizare GPS sau platforme cloud, adresează parțial aceste neajunsuri, însă introduc alte limitări: dependența de conexiunea la internet, riscul falsificării locației sau preocupări legate de confidențialitatea datelor personale.

Lucrarea mea propune ca soluție PresSync — un sistem autonom de gestionare a prezenței, care utilizează infrastructura Wi-Fi locală ca mecanism de validare a prezenței fizice.

1.1 Structura lucrării

Lucrarea este structurată în cinci capitole, organizate progresiv de la analiza problemei până la implementarea și evaluarea soluției propuse.

- ▷ **Capitolul 1 — Introducere** oferă o imagine de ansamblu asupra contextului care a motivat dezvoltarea sistemului PresSync, prezintă pe scurt soluția propusă și descrie structura lucrării.
- ▷ **Capitolul 2 — Definirea problemei** analizează metodele actuale de pontaj și limitările acestora, realizează o comparație structurată a soluțiilor existente pe piață și fundamentează arhitectural alegerea abordării bazate pe conectivitate Wi-Fi la nivelul Stratului 2. De asemenea, capitolul definește obiectivele sistemului, actorii implicați și scenariile de bază de utilizare.
- ▷ **Capitolul 3 — Analiza software** prezintă cerințele funcționale și non-funcționale ale aplicației, modelul funcțional al sistemului, descrierea detaliată a cazurilor de utilizare și diagramele UML asociate.
- ▷ **Capitolul 4 — Proiectare și implementare** descrie arhitectura tehnică a soluției, incluzând proiectarea bazei de date, tehnologiile utilizate, diagramele de interacțiune și de clase, precum și secvențe reprezentative de cod sursă.
- ▷ **Capitolul 5 — Concluzii** sintetizează contribuțiile proprii aduse prin această lucrare, evaluează rezultatele obținute în raport cu obiectivele definite și propune direcții de dezvoltare ulterioară a sistemului.

Capitolul 2

Definirea problemei

Acest capitol definește problema gestionării prezenței în diferite contexte și oferă o imagine de ansamblu asupra sistemelor actuale de pontaj și a limitărilor acestora. De asemenea, este realizată o analiză comparativă a modelelor existente cu soluția propusă de mine și se enumeră obiectivele acesteia. Capitolul se încheie cu definirea actorilor și sistemului și prezentarea scenariilor de bază de utilizare.

2.1 Introducere și Contextualizare

Dacă ai participat vreodată la un curs, o ședință sau o conferință, știi că momentul în care se face prezența este o pierdere de timp. Listele pe hârtie circulate prin sală, strigarea fiecărui nume sau distribuirea cartelelor de acces sunt soluții banale, dar ele ascund probleme reale: timp pierdut, erori frecvente și participanți care sunt pe listă deși nu se află în cameră.

Această situație, unde un coleg îl marchează pe altul prezent deși nu e adevărat, nu este doar o nedreptate față de ceilalți, dar creează și o greșală în datele colectate. Acest lucru poate afecta decizii ce țin de evaluare sau de monitorizarea activității. Această problemă nu este una trivială, deoarece, cu cât grupul este mai mare cu atât crește riscul ca astfel de situații să treacă neobservate.

Este important să evidențiem că această problemă nu aparține doar mediului universitar, chiar dacă soluția mea a pornit de acolo. Ea poate fi aplicată și în medii cum ar fi o conferință sau orice alt tip de eveniment unde organizatorul ar avea nevoie să știe exact numărul de participanți și cine sunt ei, pentru a crea mai apoi statistici sau rapoarte pentru planificarea următorului eveniment. Deci, cum verificăm că oamenii sunt cu adevărat acolo, fără a întrerupe activitatea și fără a permite date inexacte? Avem nevoie de un mecanism de înregistrare a prezenței care să fie fiabil, rapid și capabil să genereze informațiile utile, nu o simplă listă bifată în grabă

2.2 Analiza soluțiilor existente

Pe piață există deja o varietate largă de soluții care gestionează prezența. De la liste de hârtie, carduri fizice până la platforme cloud sofisticate. Problema nu este lipsa soluțiilor, ci faptul că multe din ele nu rezolvă în totalitate problema.

Unele sisteme necesită costuri suplimentare pentru a fi implementate. Echipamente hardware dedicate și o infrastructură tehnică. Alte soluții sunt mai accesibile dar nu oferă garanția că persoana se află fizic în încăpere.

În toate mediile, nevoia principală rămâne aceeași: o soluție ușor de folosit, de implementat și care nu permite fraudă. Acest echilibru este ceea ce lipsește majorității soluțiilor. Pentru a înțelege exact unde se situează PresSync, urmează să prezint o comparație cu cele trei tipuri de sisteme care sunt cel mai des utilizate în prezent.

2.2.1 Sistemele cu cartele magnetice

Cartelele magnetice sunt probabil cele mai cunoscute soluții de control al accesului pe care le putem întâlni. Funcționarea lor se bazează pe principiul că participantul deține un obiect fizic, pe care îl prezintă unui cititor la intrarea în spațiu. Prezența și autentificarea au loc instant. Simplitatea este punctul forte al acestui tip de soluții: scanarea rapidă, infrastructura conectată prin cablu la serverele instituției și funcționează într-o rețea izolată.

Problema principală o constituie costurile care sunt semnificative: cumpărarea și instalarea cititoarelor la fiecare intrare, mentenanța continuă și înlocuirea periodică a cartelelor pierdute sau deteriorate. De asemenea, sistemul este ancorat de acea zonă. Mai mult de atât există și riscul ca un coleg să îți dea cartela altuia pentru a fi marcat prezent.

2.2.2 Sisteme mobile bazate pe scanare de coduri QR

Codul QR afișat pe un videoproiector a devenit ceva familiar în sălile de curs în ultimii ani. Se generează un cod QR, care mai apoi este proiectat, iar studenții scanează codul cu telefonul. Implementarea nu necesită hardware suplimentar, sistemul se folosește de tehnologia existentă deja și poate fi pus în funcțiune în câteva minute.

Această abordare este una ideală în contextul educațional unde bugetul este limitat. Însă problema acestei soluții este securitatea, deoarece ea se bazează pe faptul că doar persoanele

prezențe fizic au acces la cod, însă această ipoteză nu poate fi adevărată. Un singur participant din sală poate fotografia codul și îl trimite pe WhatsApp oricui. Nici versiunile cu coduri dinamice nu rezolvă complet problema: codul poate fi partajat în timp real.

2.2.3 Sisteme de tracking pe bază de localizare GPS

Aceste platforme au o abordare diferită. Ele verifică dacă participantul se află în locul potrivit. Utilizatorul are nevoie doar să facă un check-in din aplicație, iar sistemul compară coordonatele GPS ale telefonului cu o zonă predefinită în aplicație.

Această soluție este flexibilă, nu are nevoie de o infrastructură hardware, evenimentul poate fi configurat oriunde. Totuși pentru săli de curs, birouri etc., această abordare are trei probleme.

Prima este falsificarea coordonatelor GPS. Există aplicații specializate care permit simularea unei locații. A doua problemă este confidențialitatea. Și a treia limitare este de natură tehnică, sistemul are nevoie de o conexiune constantă la internet. În spații cu semnal slab aplicația devine inoperabilă.

2.2.4 Sinteza comparativa

Criteriu comparativ	Sisteme RFID / Cartele	Scanare Cod QR	Tracking GPS (Cloud)	PresSync
Costuri de implementare	Ridicate (necesită hardware dedicat și cartele)	Foarte scăzute (folosește infrastructura mobilă existentă)	Medii spre ridicate (abonamente SaaS/Cloud)	Scăzute (necesită un router compatibil OpenWRT și/sau un sistem embedded)
Risc de fraudă	Ridicat (delegarea fizică a token-ului de autentificare)	Foarte ridicat (redistribuirea codului optic în afara perimetrului)	Ridicat (falsificarea coordonatelor GPS prin location mocking)	Scăzut (conectivitatea la nivelul Stratului 2 este condiționată de atenuarea fizică a semnalului Wi-Fi)
Versatilitate	Redusă (infrastructură fixă, instalată permanent)	Ridică	Foarte ridicată	Ridică (echipament portabil, instalabil în orice spațiu cu alimentare electrică)
Dependență de conexiune externă	Nu (funcționează de regulă în rețea Intranet)	Da	Da (obligatoriu)	Nu (sistemul funcționează într-un mediu de rețea complet izolat, fără conexiune externă)

Tabelul 2.1: Comparatie între principalele soluții de gestionare a prezenței

2.2.5 Fundamentarea arhitecturală a soluției PresSync

Analizând comparația, putem să ne dăm seama că fiecare abordare expune o problemă, fie securitatea, fie accesibilitatea economică. PresSync a fost conceput pentru a rezolva toate aceste probleme prin valorificarea proprietății fizice a semnalului Wi-Fi.

Primul pas este înlocuirea cartelei cu dispozitivul mobil personal, ceea ce practic rezolvă împrumutarea token-ului unui coleg. Identitatea digitală a participantului este stocată sub forma unui token JWT, legat de contul său și accesibil exclusiv prin autentificare, ceea ce transformă dispozitivul într-un factor de autentificare.

Elementul principal al arhitecturii PresSync, și diferență cea mai mare de soluțiile analizate mai sus, este folosirea proprietăților fizice ale undelor radio ca barieră antifraudă. Această abordare este mai robustă, deoarece validarea prezenței se face prin conectivitate la nivelul Stratului 2 (Data link layer) al modelului OSI. Routerul este configurat în regim de portal

captiv , emite pe benzile 2,4GHz și 5GHz, frecvențe care sunt sensibile la obstacole fizice din mediul înconjurător.

Atunci când traversează materiale de construcție uzuale , cum ar fi pereți din beton , cărămidă sau gips-carton , semnalul suferă o atenuare destul de mare. Ceea ce face ca semnalul emis să fie insuficient pentru ca un dispozitiv să se conecteze de la câțiva metri de punctul de acces.Prin urmare , pentru a transmite token-ul JWT și a înregistra prezența , telefonul trebuie să se afle fizic în interiorul spațiului delimitat de router.

Încă un factor care contribuie la această restricție naturală este zona Fresnel. Obstacolele solide intersectate de această zonă reduc puterea semnalului.Combinând acești doi factori , transformăm pereții încăperii într-o barieră naturală antifraudă.

Acest ultim factor previne atacurile de tip MAC spoofing care necesită prezența fizică în raza de acțiune a routerului și echipament specializat.

Încă un factor important al sistemului este independență față de conexiunea la internet . Aplicația rulează local , nu există servere externe , abonament cloud etc. Această alegere aduce două consecințe practice : Prima este ca sistemul funcționează în orice spațiu cu alimentare electrică.A doua este ca datele sunt procesate local fără a fi transmise la alte servicii.

În concluzie PresSync rezolva problema la un nivel mai profund.Ancorând validarea prezenței în proprietăți fizice și eliminând dependență de infrastructura externă.

2.3 Descrierea problemei și soluția propusă

Pe baza secțiunilor prezentate mai sus , putem formula problema principală pe care sistemul o abordează: necesitatea unui mecanism de înregistrare a prezenței care să ofere certitudinea prezenței fizice reale a participanților, fără prea multe costuri sau dependente de conexiunea externă și păstrarea datelor local. Având în vedere deficiențele soluțiilor deja existente menționate anterior , am considerat necesar construirea unei soluții gazdite local pe un router OpenWRT , în care prezența fizică în rețea este dovada participării.Ceea ce elimină dependență de internet sau servicii GPS.PresSync automatizează pontajul , printr-o metoda sigură , rapidă și utilă.Contextele aplicării pot fi variate , de la mediul academic până la conferințe și evenimente temporare

2.4 Obiectivele sistemului

Pentru a materializa viziunea descrisa mai sus , am fost ghidat de urmatoare obiective functionale și tehnice:

- O1. Funcționare autonomă și independență hardware.** Sistemul este proiectat să ruleze izolat pe echipamente accesibile, precum un router OpenWRT. Această abordare elimină complet dependența de o conexiune la internet sau de servere , asigurând un mediu securizat și în care prezența în rețeaua fizică constituie dovada fundamentală a participării.
- O2. Automatizarea calendarului și a recurenței.** sistemul preia munca repetitivă a administratorului. Odată definită o categorie de evenimente împreună cu regulile sale de recurență (zilnic, săptămânal etc.), aplicația instanțiază automat noi evenimente la momentul oportun, fără nicio intervenție manuală.
- O3. Securizarea și delimitarea strictă a accesului.** Autentificarea utilizatorilor este realizată prin *token*-uri JWT, asigurând un flux clar de autorizare. Fiecare actor are vizibilitate limitată la resursele relevante: participantul își confirmă prezența și își verifică istoricul personal, în timp ce organizatorul administrează resursele colective și accesează datele întregului grup.
- O4. Procesare concurentă și scalabilitate ridicată.** Sistemul separă operațiile de modificare a datelor (comenzi) de cele de citire. Această separare asigură că cererile de marcarea a prezenței sunt procesate instantaneu și fără conflicte, iar cererile complexe de calcul al statisticilor rulează pe un flux propriu optimizat, fără a bloca aplicația.
- O5. Generarea de statistici în timp real.** În loc să stocheze datele în formă brută, sistemul le transformă în informații utile la cerere. Acesta calculează procentajul de prezență la nivel de utilizator sau eveniment și permite extragerea de rapoarte detaliate direct din interfață.

2.5 Actori principali și delimitarea rolurilor

Sistemul PresSync lucrează cu două categorii principale de utilizatori, pe care le putem considera *stakeholder*-ii primari și secundari ai platformei.

Administratorul (Organizatorul) reprezintă *stakeholder*-ul primar. El este responsabil de configurarea evenimentelor și a categoriilor acestora, de gestionarea participanților, de stabilirea regulilor de recurență (atunci când evenimentele se repetă) și de generarea rapoartelor statistice. Acest rol are acces complet la toate funcționalitățile de administrare ale sistemului.

Participantul este *stakeholder*-ul secundar. El interacționează cu platforma doar pentru a-și

marca prezența la evenimentele la care este înscris și pentru a consulta propriul istoric de participare. Accesul său este strict limitat la informațiile și acțiunile care îl privesc direct.

Această separare clară între roluri nu este întâmplătoare. Ea asigură atât o protecție mai bună a datelor, cât și o experiență adaptată nevoilor fiecărui tip de utilizator. Totodată, respectă principiul privilegiului minim (*principle of least privilege*), conform căruia un utilizator ar trebui să aibă doar accesul strict necesar pentru a-și îndeplini atribuțiile.

2.6 Scenarii de baza

Pentru o mai bună înțelegere a sistemului voi explica pe pași cum are loc procesul de marcare a prezenței , creare evenimente , vizualizare prezențe și administrare utilizatori.

Pentru acest exemplu vom folosi mediul academic.

- ▷ Profesorul Vlad Moraru , având deja sistemul activ și fiind conectat la rețea, deschide o pagină web și este redirectionat de către portalul captiv pe pagina aplicației PresSync.Se înregistrează sau se autentifică dacă are deja cont , fiind moderator/administrator are nevoie de un cod unic care este transmis pe emailul introdus.
- ▷ După completarea formularului este redirectionat pe pagina principală și la apăsarea butonului "Crează categorie de eveniment ", este afișat un formular ,unde introduce datele despre categoria de eveniment pe care vrea să o creeze.
- ▷ Sistemul folosește categoria de eveniment ca un template și apoi crează un eveniment la data și ora setată.
- ▷ Studentul ,aflat în sala de curs și fiind conectat la rețeaua oferită de router, deschide browserul și este redirectionat către pagina de autentificare , completează formularul și dacă există un eveniment activ și prezența este deschisă sistemul îl pune ca prezent. Fără să apese nici un buton în plus.
- ▷ Pentru a viziona prezența ambii au posibilitatea de a căuta categoria de eveniment dorită și pentru fiecare informațiile vor fi diferite , pentru student va fi afișat doar propriile prezențe și pentru ceilalți actori vor fi afișate prezențele la toți participanții

Capitolul 3

Analiza Software a Sistemului

Analiza software a sistemului este crearea modelelor conceptuale, ce ne ajuta să abstractizam o situație sau un concept. Cu ajutorul ei putem să identificăm riscuri sau neajunsuri ale unui sistem și să ne dam seama despre cum ar trebui să arate soluția în finalitate. Aici voi prezenta documentul de cerințe, diagrama de clase a modelului domeniului și diagrame de secvențe de sistem. În documentul de cerințe avem actorii care participa la functionarea sistemului și modurile în care sistemul poate fi folosit. Diagrama de secvențe de sistem ne ilustrează modul în care obiectele comunica între ele, mesaje și ordinea lor.

3.1 Documentul de cerințe

Platforma PresSync a fost gândită ca o soluție completă pentru monitorizarea în timp real a prezenței la diverse activități și evenimente. Ea integrează mai multe module care lucrează împreună, urmărind atât fluxurile utilizatorilor, cât și managementul evenimentelor. În esență, sistemul centralizează tot ce ține de conturi, configurarea activităților, înscrierea participanților și validarea prezenței, combinând automatizări cu posibilitatea de intervenție manuală acolo unde este necesar.

3.1.1 Actori și roluri în sistem

Admin – Are control total asupra platformei. El poate gestiona conturile de Moderator, configura permisiunile globale de securitate și are acces la toate setările critice ale sistemului.

Moderator

– Se ocupă de partea operațională zilnică. El creează și gestionează evenimentele, coordonează utilizatorii înscriși și poate genera rapoarte de prezență sau audit.

User – Reprezintă participantul obișnuit. Poate să se înscrie la evenimente, să-și vadă istoricul personal de participare și să acceseze informațiile relevante pentru el.

3.1.2 Cerințe funcționale

F1. Gestiunea fluxurilor de înregistrare și autentificare

- F1.1** Utilizatorii se pot înregistra în sistem prin formulare dedicate, în funcție de rolul ales (Admin, Moderator sau User).
- F1.2** Datele introduse sunt verificate atât din punct de vedere sintactic, cât și semantic. Dacă apar erori, utilizatorul vede alerte clare, dar informațiile completate corect nu se pierd.
- F1.3** După finalizarea cu succes a înregistrării, sistemul creează automat contul și redirecționează utilizatorul către pagina de profil.
- F1.4** Autentificarea se face prin nume de utilizator și parolă. În funcție de rol, interfața se ajustează dinamic, afișând doar funcționalitățile permise.
- F1.5** Dacă utilizatorul renunță la procesul de login sau înregistrare, este redirecționat automat către pagina principală.

F2. Managementul ciclului de viață al evenimentelor

- F2.1** Un eveniment nou poate fi creat prin completarea unor informații esențiale: titlu, perioadă, locație, descriere și tipul activității.
- F2.2** Moderatorii pot modifica detaliile unui eveniment oricând, iar ștergerea definitivă se face doar după o confirmare explicită, pentru a evita greșeli.
- F2.3** Există o listă centralizată a tuturor evenimentelor, cu posibilitatea de a vizualiza detaliile complete ale fiecăruia printr-un simplu click.

F3. Administrarea conturilor de utilizator

- F3.1** Crearea și configurarea conturilor de Admin este permisă doar utilizatorului cu drepturi maxime (Super Admin).
- F3.2** Sistemul permite blocarea temporară sau reactivarea conturilor atunci când este necesar (de exemplu, în caz de inactivitate îndelungată sau probleme de securitate).
- F3.3** Atât Adminul cât și Moderatorii au acces la o secțiune centralizată unde pot vedea listă completă a utilizatorilor și detaliile profilurilor acestora.

F4. Validarea și monitorizarea prezenței

- F4.1** Pentru a marca prezența, trebuie mai întâi selectată o activitate din catalog.
- F4.2** Prezența se poate înregistra atât manual de către organizator, cât și automatizat (unde este posibil), evitându-se astfel înregistrările duble pentru același eveniment.
- F4.3** În orice moment se poate vedea o situație centralizată, clar separată între participanții prezenți și cei absenți.

F5. Module de raportare și analiză

- F5.1** Sistemul agregă datele de prezență și generează diverse statistici utile privind gradul de participare la activități.
- F5.2** Rapoartele pot fi exportate atât în format PDF, cât și CSV, pentru a putea fi prelucrate ulterior.
- F5.3** Utilizatorii simpli au acces la propriul istoric de participări, afișat cronologic.

3.1.3 Cerințe nefuncționale

- CN1. Ergonomie și usabilitate** – Interfața trebuie să fie intuitivă și modernă, astfel încât un utilizator nou să se poată obișnui rapid cu platforma, indiferent de nivelul tehnic.
- CN2. Toleranță la erori și robustețe** – Aplicația va trata cu atenție erorile, oferind mesaje clare și prietenoase, fără termeni tehnici care să confuzeze utilizatorul.
- CN3. Mediu de dezvoltare și tehnologii** – Partea de backend va fi dezvoltată în Java folosind o arhitectură stratificată, ușor de întreținut și extins.
- CN4. Portabilitate** – Soluția trebuie să ruleze fără probleme pe Windows, Linux și macOS.
- CN5. Securitate și confidențialitate** – Parolele vor fi stocate hashed cu algoritmi puternici, iar accesul la funcționalități se va face prin RBAC (control granular pe roluri).

3.2 Cazuri de utilizare

3.2.1 Creare Cont

Descriere: Procesul complet de înregistrare a unui utilizator nou în sistem. Utilizatorul completează datele personale, iar sistemul creează contul, criptează parola și trimite un cod OTP pe email pentru verificare înainte ca acesta să poată fi folosit.

Actor: Utilizator (principal)

Eveniment declanșator: Utilizatorul accesează pagina de înregistrare și trimite formularul cu datele personale.

Precondiții: Sistemul este operațional. Adresa de email nu este deja asociată unui cont existent.

Postcondiții: Contul este creat în baza de date (inițial inactiv), parola este stocată securizat folosind BCrypt, iar un cod OTP este trimis pe email pentru activare.

Flux principal:

Utilizator	Sistem
1. Cere crearea unui cont	2. Afișează formularul de înregistrare
3. Completează datele	4. Verifică datele primite (sintactic și semantic) [A1][A2]
	5. Creează cont nou și stochează parola securizat
	6. Memorează datele contului creat
	7. Afișează mesaj de succes

Fluxuri alternative:

[A1] Datele sunt introduse eronat Sistemul afișează un mesaj de eroare clar și marchează câmpurile greșite. Utilizatorul poate corecta informațiile fără a pierde ce a completat deja.

[A2] Utilizatorul există deja (eroare semantică): Sistemul detectează că username-ul sau email-ul este deja folosit, afișează mesajul „Contul există deja” și revine la formular.

3.2.2 Vizualizare Prezență cu Statistică

Descriere: Utilizatorul vizualizează istoricul propriu de prezență și primește statistici personale (inclusiv predicții). Adminul poate vizualiza statistici agregate, aplica filtre și genera rapoarte detaliate.

Actor: Utilizator / Admin (principal)

Eveniment declanșator: Actorul selectează opțiunea „Statistici” sau „Rapoarte” din meniul principal.

Precondiții: Actorul este autentificat și există date de prezență înregistrate.

Postcondiții: Sistemul afișează datele solicitate sub formă de tabel și/sau grafice.

Flux principal (Admin):

Admin	Sistem
1. Selectează „Rapoarte și Statistici”	2. Afișează dashboard-ul cu statistici agregate
3. Aplică un filtru (perioadă, eveniment, grupă etc.)	4. Validează criteriile de filtrare [A1]
	5. Afișează listă de rezultate
6. Selectează un utilizator sau eveniment	7. Generează raportul detaliat
8. Selectează „Descarcă Raport”	9. Generează fișierul (PDF/CSV) [A2]
	10. Afișează mesaj de succes

Flux principal (Utilizator):

Utilizator	Sistem
1. Selectează „Statistici”	2. Aplică automat filtrul pe ID-ul propriu
	3. Generează și afișează istoricul de prezență
4. Selectează un eveniment specific	5. Afișează detaliile evenimentului

Fluxuri alternative:

[A1] **Criterii de filtrare invalide:** Sistemul afișează eroare și marchează câmpurile greșite.

[A2] **Eșec la generarea raportului:** Sistemul afișează un mesaj clar de eroare tehnică.

[A3] **Nu există date:** Sistemul afișează mesajul „Nu au fost găsite date de prezență înregistrate.”

3.2.3 Înregistrare Prezență

Descriere: Procesul prin care un utilizator își înregistrează prezența la un eveniment activ. Sistemul verifică existența evenimentului și dreptul de acces al utilizatorului.

Actor: Utilizator (principal)

Eveniment declanșator: Utilizatorul accesează funcția de prezență sau se autentifică în timpul unui eveniment activ.

Precondiții: Există un eveniment activ la care utilizatorul nu are deja prezența înregistrată și nu este restricționat.

Postcondiții: Sistemul salvează prezența utilizatorului la evenimentul activ.

Flux principal:

Utilizator	Sistem
1. Cere înregistrarea prezenței	2. Verifică evenimente active și drepturi [A1] 3. Afișează detaliile evenimentului curent
4. Confirmă prezența [A2]	5. Salvează prezența 6. Afișează mesaj de înregistrare completă

Fluxuri alternative:

[A1] **Nu există eveniment activ sau utilizatorul este restricționat** Sistemul afișează statistică prezențelor înregistrate și fluxul se termină.

[A2] **Utilizatorul refuză/amână confirmarea** Utilizatorul apasă „Nu acum” sau „Anulează”. Sistemul îl redirecționează către ecranul principal.

3.2.4 Administrează Prezență Manual

Descriere: Adminul poate modifica manual prezența unui utilizator la un eveniment (de exemplu, dacă a uitat să se marcheze).

Actor: Admin (principal)

Eveniment declanșator: Adminul selectează un eveniment pentru administrare.

Precondiții: Evenimentul există și adminul este autentificat.

Postcondiții: Prezența este actualizată și statisticile recalulate.

Flux principal:

Admin	Sistem
1. Selectează evenimentul	2. Afișează listă participanților și statusul lor
3. Modifică starea unui utilizator	4. Validează acțiunea
	5. Salvează modificarea
	6. Recalculează statisticile
	7. Afișează listă actualizată

3.2.5 Creare Categorii de Evenimente

Nume: Creare categorii de evenimente

Descriere: Adminul definește o nouă categorie (Curs, Seminar, Laborator etc.) cu setări de orar, fereastră de prezență și reguli de recurență.

Actor: Admin (principal)

Flux principal:

Admin	Sistem
1. Selectează „Creare categorie”	2. Afișează formularul
2. Completează datele	3. Verifică datele [A1]
	4. Salvează categoria
	5. Afișează mesaj de succes

3.2.6 Creare Evenimente

Descriere: Adminul creează o instanță concretă a unei categorii pentru o anumită dată, cu opțiuni de recurență.

Actor: Admin (principal) **Flux principal:**

Admin	Sistem
1. Selectează „Creare eveniment”	2. Afișează formularul
2. Completează datele	3. Validează datele [A1][A2]
	4. Salvează evenimentul
	5. Afișează mesaj de succes

Fluxuri alternative:

[A1] **Date invalide:** Sistemul marchează câmpurile greșite.

[A2] **Conflict de orar:** Sistemul avertizează despre suprapunere și permite modificarea.

De asemenea, aici trebuie să includem și diagrama cazurilor de utilizare care reprezintă actorii, cazurile de utilizare și legăturile dintre toate elementele. Tipurile de relații sunt prezentate în tabelul de mai jos.

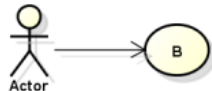
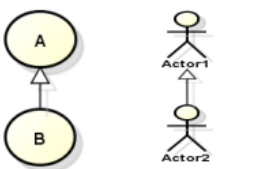
Denumire	Semantica	Sintaxa concreta UML
Relația de asociere	Describe comunicarea dintre actori și sistem	 powered by astah
Relația de dependență (include)	Ori de câte ori va fi executat cazul A va fi executat și cazul B	 powered by astah
Relația de dependență (extend)	Dacă se execută cazul A și este îndeplinită o condiție se execută și cazul B	 powered by astah
Relația de generalizare	Are loc între actori sau între cazuri de utilizare	 powered by astah

Figura 3.1: Tipuri de relații în diagrama de cazuri de utilizare

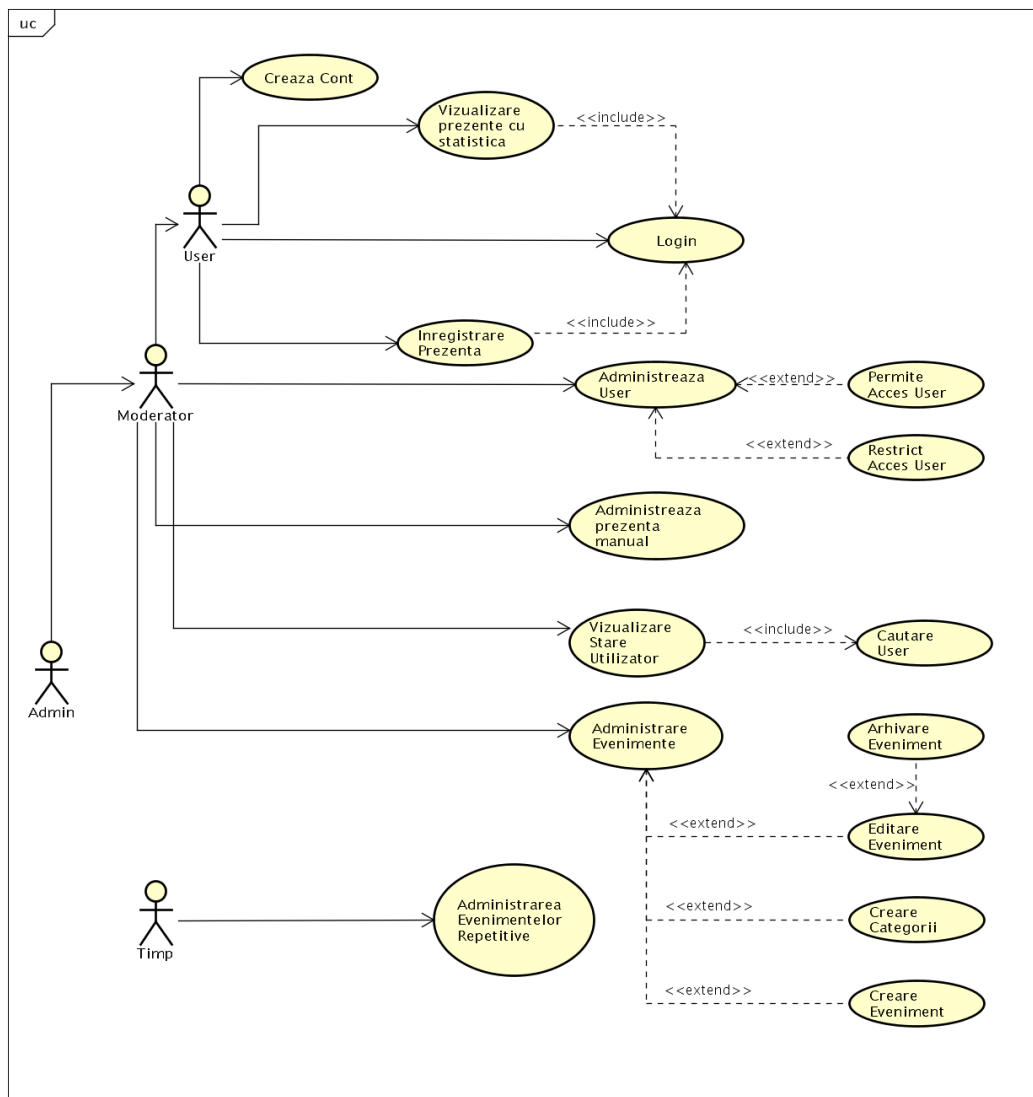
3.3 Diagrame de secvențe de sistem

3.3.1 Evenimente de sistem

Evenimentele de sistem apar datorită acțiunilor actorilor asupra sistemului software. Ele sunt de trei tipuri:

- ▷ De flux de date (create de acțiuni ce transmit date sau informații)
- ▷ De control (nu conțin date)
- ▷ Temporare (evenimente generate de un cronjob)

După ce am analizat cazurile de utilizare am dedus următoarele evenimente de sistem:



powered by Astah

Figura 3.2: Diagrama cazurilor de utilizare

Tip	Nr.	Descriere
Flux de date	1	Utilizatorul introduce datele de înregistrare sau logare și le trimite sistemului
Flux de date	2	Utilizatorul introduce codul primit pe adresa de mail electronică și îl trimite sistemului
Flux de date	3	Utilizatorul introduce numele unei categorii de eveniment în bara de căutare și trimite sistemului
Control	4	Sistemul marchează utilizatorul ca fiind prezent la evenimentul activ
Control	5	Sistemul autorizează accesul utilizatorului
Control	6	Sistemul verifică datele introduse de utilizator
Temporare	7	Sistemul creează evenimente în baza timpului setat de administrator
Temporare	8	Sistemul deschide prezența la evenimente dacă fereastra de prezență coincide cu timpul actual

Tabelul 3.1: Evenimente de sistem identificate

3.3.2 Diagrame de secvență de sistem

Aceste diagrame descriu interacțiunea dintre actori și sistemul software. Interacțiunea e descrisă de mesaje, care la rândul lor reprezintă comunicarea dintre două obiecte. Urmează să prezint conceptele diagramei de secvențe și a sintaxei în limbajul UML.

Așadar, acestea sunt diagramele de secvență de sistem pe care le-am făcut în urma analizei soluției mele.

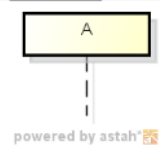
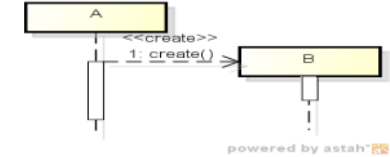
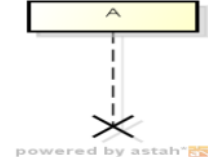
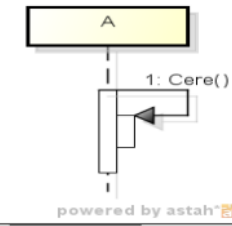
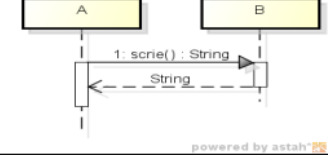
Linia de viață - indică existența unui obiect pe o perioadă de timp	
Mesajul <<create>> - în această perioadă este creat un obiect	
Mesajul <<destroy>> - descrie distrugerea unui obiect	
Activare - indică executarea unei operații de către obiectul însuși sau prin intermediul altor obiecte	
Mesajul <<return>> - indica tipul de date primit	

Figura 3.3: Prezentarea diagramelor de secvență de sistem

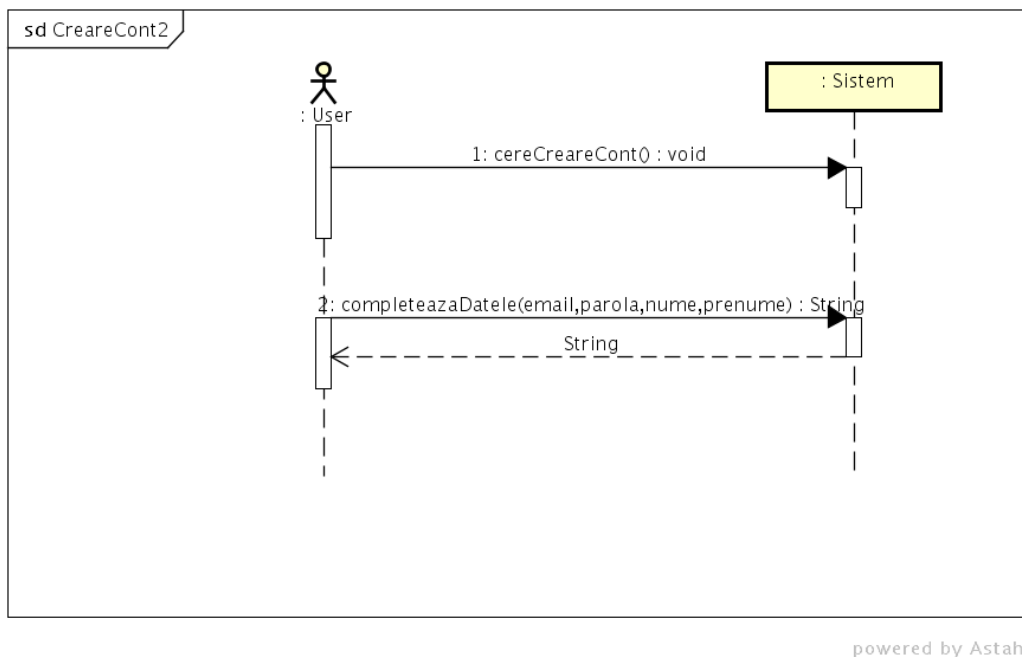
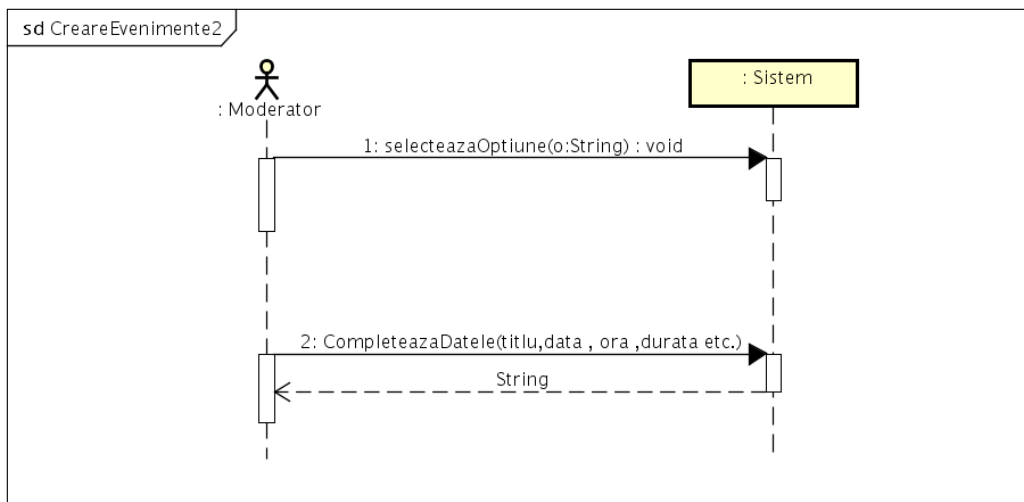
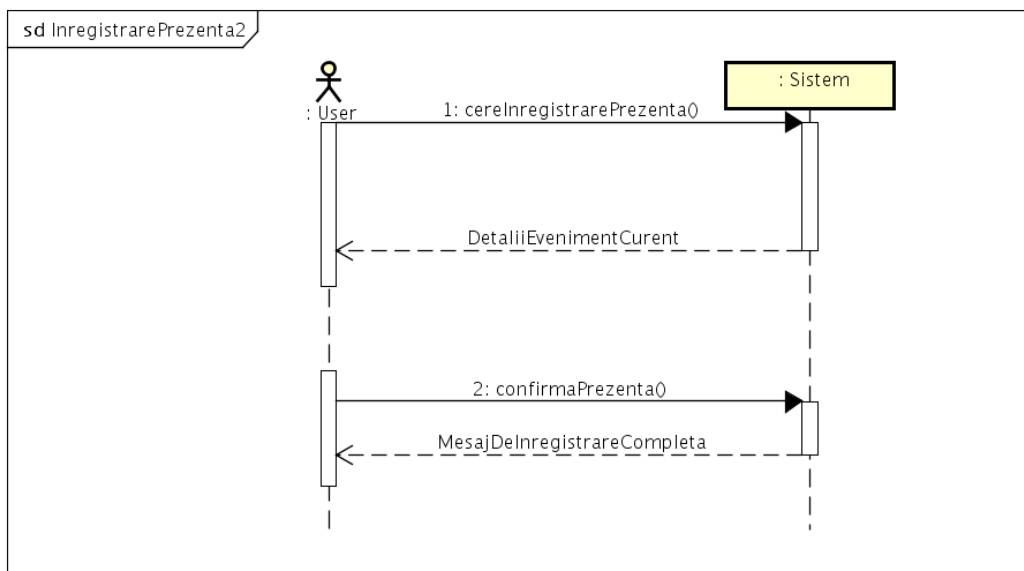


Figura 3.4: Diagrama de secvență — Creare cont



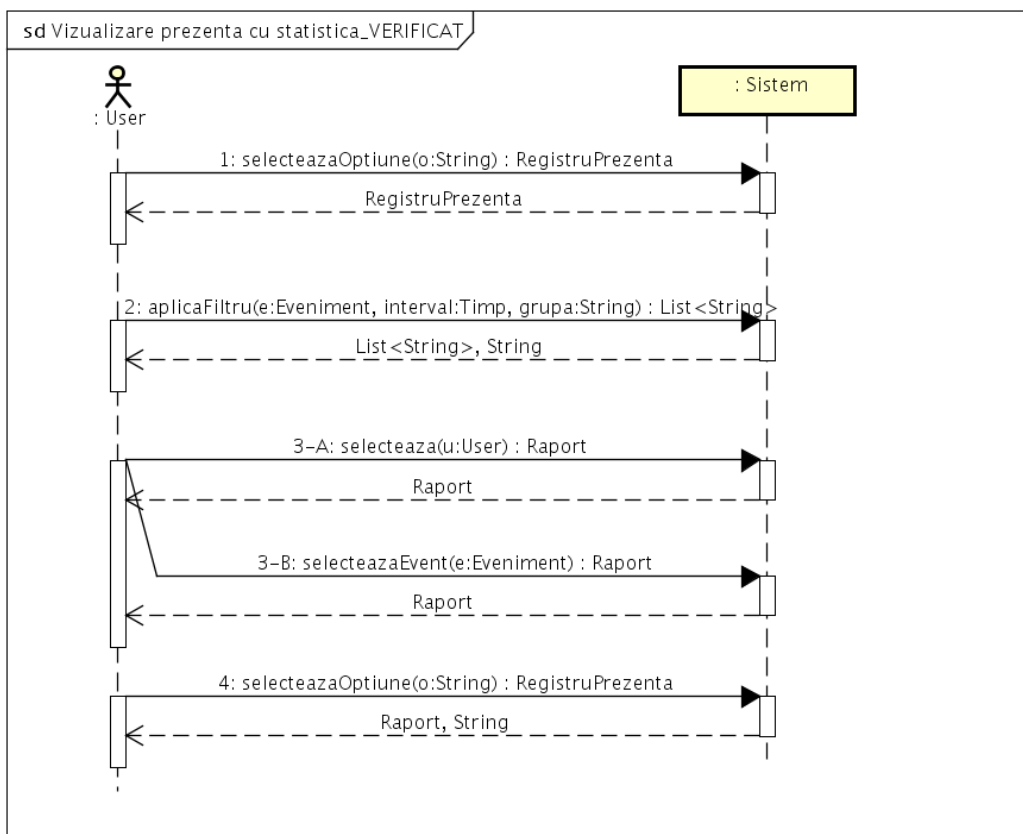
powered by Astah

Figura 3.5: Diagrama de secvență — Creare evenimente



powered by Astah

Figura 3.6: Diagrama de secvență — Înregistrare prezență

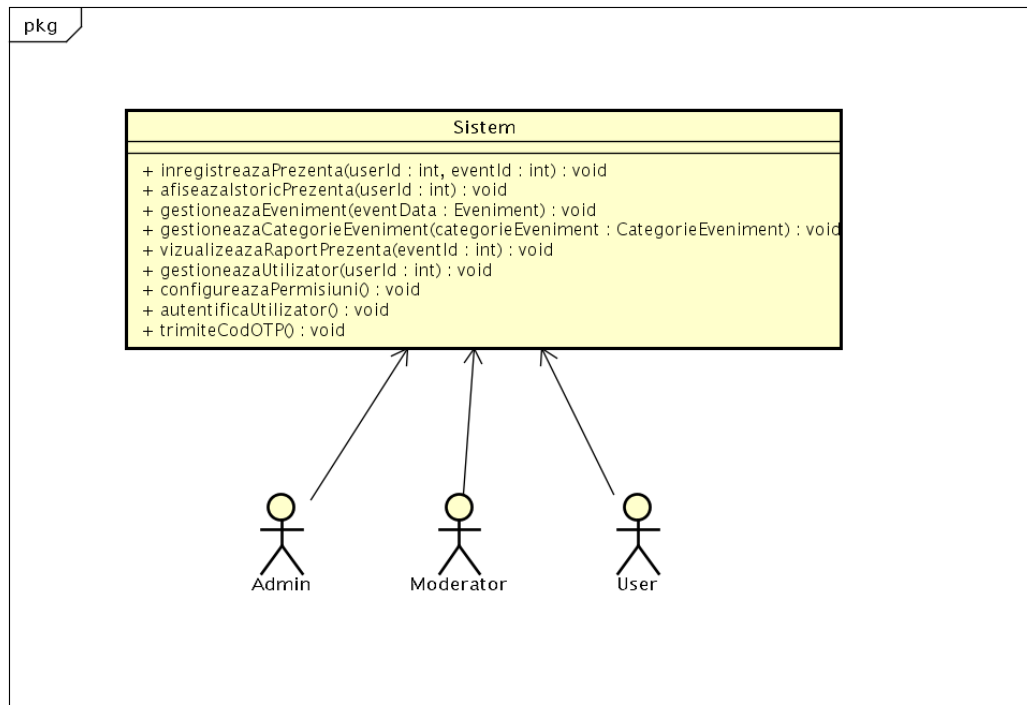


powered by Astah

Figura 3.7: Diagrama de secvență — Vizualizare prezență

//start processing from here

Diagrama de context a sistemului este descrisa de actiunile actorilor asupra sistemului si operatiile efectuate de acesta.



powered by Astah

Figura 3.8: Diagrama de context

3.4 Modelul domeniului

Crearea modelului de domeniu este un alt pas important care are la baza diagrama de clase. Diagrama de Clase este bazata pe urmatoarele concepte: Clasele sunt reprezentarea unui grup cu atribute similare si cu operatii similare. Atributele sunt informatii detinute de clase si pot lua anumite valori. Operatiile sunt functii sau schimbari ce pot fi aplicate.

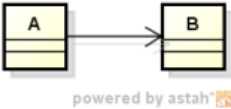
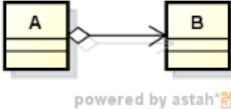
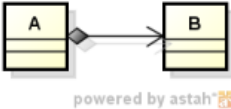
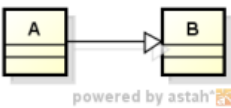
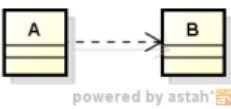
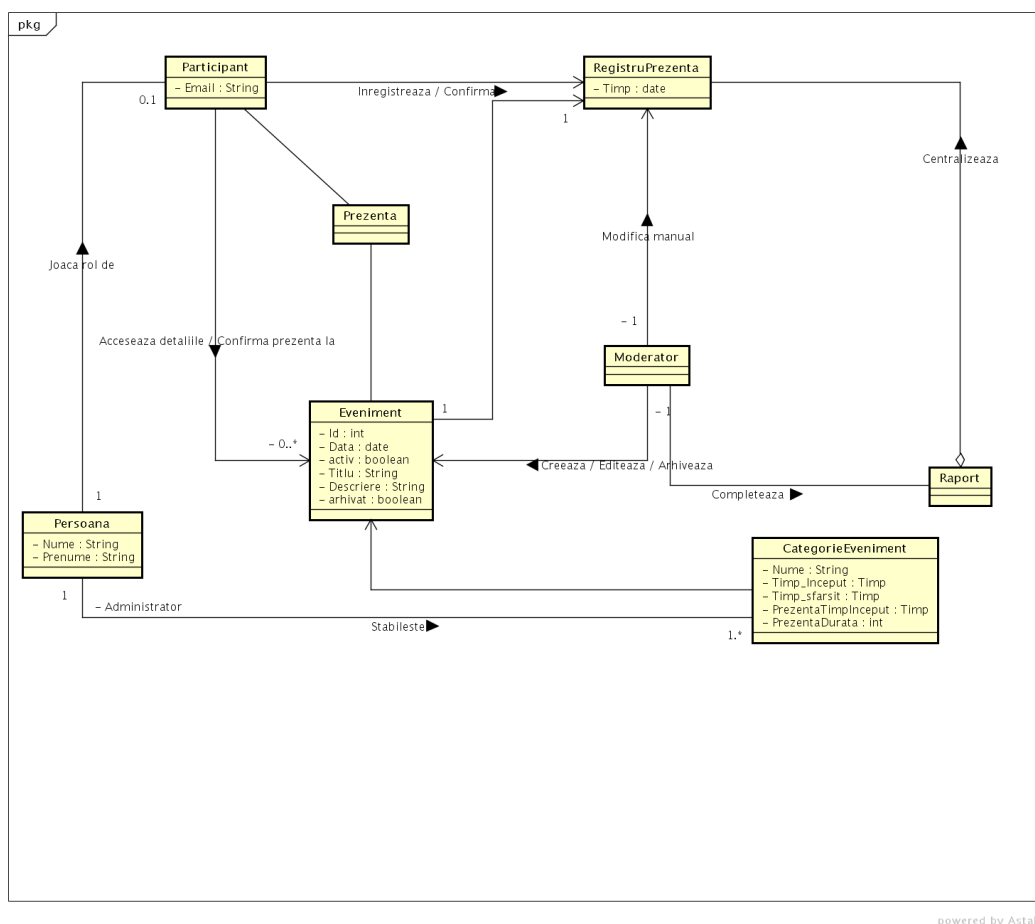
Relația de asociere	Reprezintă mulțimea de legături între obiectele claselor implicate	
Relația de agregare	Reprezintă formarea unui lucru din părțile sale	
Relația de compoziție	Este o relație de agregare mai puternică, deoarece un obiect nu poate exista fără celelalte obiecte agregat de care este legat.	
Relația de generalizare /specializare	Face legătura între două clase în care una este supraclasa și cealalta este subclasa	
Relația de dependență	Relație prin care dacă se modifică o clasa duce la afectarea altei clase cu care este legată	

Figura 3.9: Explicarea relațiilor în modelul domeniului

Pentru a realiza diagrama trebuie sa identificam substantivele din descrierea problemei , ca mai apoi sa le transformam in clase sau atribute. Astfel , dupa analiza descrierii problemei am construit digrama de clase prezentata mai jos.



powered by Astah

Figura 3.10: Diagrama de clase a modelului domeniului

Capitolul 4

Proiectarea si implementarea aplicatiei

Inca o activitate de baza , si foarte importanta atunci cand construim o solutie la nivel conceptual. Acest stagiou este o urmare identificarii si descrierii cerintelor pe care le am identificat mai sus.Tot aici construim arhitectura software a sistemului.

4.1 Arhitecturi si stiluir arhitecturale

Aici reprezentam modelul de organizare a sistemelor software.Modelul este compus din componentele sistemului si relatiile dintre componente impreuna cu mediul in care se afla si principiile care realizeaza proiectarea si evolutia arhitecturii.

Acest stagiou ne ajuta sa intelegem constructia , managementul si evolutia unui sistem software.

Stilurile arhitecturale sunt bazate pe un vocabular de proiectare diferite constrangeri asupra modului de folosire a vocabularului si descrierea semanticii. Sunt o variatie de stiluri , in

secțiunea de mai jos voi prezenta câteva stiluri arhitecturale pe scurt.

- ▷ **Stilul arhitectural contucte si filtre** Filtrele sunt folosite pentru citirea fluxurilor de date , si conductele pentru transmiterea datelor. Acest stil este reutilizabil si permite ca mai multe procese sa fie indeplinite simultan. Dezavantajul principal este organizarea dificila a calcului.
- ▷ **Stilul arhitectural client-server** Acest stil separa procesele in doua tipuri *procese client* si *procese server*. Aceste procese colaboreaza ca sa realizeze o activitate. Avantajul este ca clientii sunt independenti si ofera un control mai stabil asupra resurselor oferite de server.
- ▷ **Stilul arhitectural bazat pe evenimente.** Comunicarea componentelor evidentiaza evenimentele. Deci o componenta transmite un semnal catre alta componenta in legatura aparitiei unui eveniment la inregistrarea acestuia.
- ▷ **Stilul arhitectural pe straturi** Acesta este structurat pe nivele , fiecare nivel ofera servicii nivelului superior si actioneaza ca client pentru nivelul inferior. Avantajul este reutilizarea usoara.
- ▷ **Stilul Model-View-Controller** Acesta se bazeaza pe izolarea componentelor logice fata de cele ce tin de interfata utilizatorului. Astfel aplicatia devine mult mai usor de modificat pentru ca nu afecteaza alte nivele.

4.2 Modele de proiectare de atribuire a responsabilitatilor

Aceste modele sunt folosite pentru a descrie principiile fundamentale de proiectare orientata spre obiecte si atribuire a responsabilitatilor claselor. Responsabilitatile sunt obligatii pe care o clasa o are. Mai jos urmeaza sa enumer cateva modele de proiectare folosite in construirea solutiei mele si problemele pe care acestea le rezolva.

Command-Query Separation (CQS) este modelul arhitectural principal al aplicației. Acesta separă operațiile de modificare a stării sistemului (comenzi) de cele de citire (interogări). Interfața `Command<E, T>` definește metoda `execute(E entity)` pentru operații de tip CREATE, UPDATE, DELETE, iar interfața `Query<I, O>` definește metoda `execute(I input)` pentru operații de tip GET. Implementări concrete precum `CreateEventCategoryCommand`, `UpdateEventCommand` și `GetAllEventsQuery` sunt organizate în pachetele `CommandHandlers` și `QueryHandlers`. Acest model asigură separarea clară a responsabilităților și previne efectele secundare în operațiile de citire, fiind o concretizare a principiului *High Cohesion* din GRASP.

Strategy Pattern este utilizat pentru gestionarea recurenței evenimentelor. Enum-ul `RepeatableType` definește strategiile suportate: DAILY, WEEKLY, BIWEEKLY, MONTHLY și YEARLY. În clasa `EventCategoryConfigService`, metoda `nextCandidateDate()` selectează

strategia corespunzătoare printr-un switch, iar în `DailyLoaderScheduler`, metoda `shouldRunToday()` decide dacă o categorie de eveniment trebuie să ruleze într-o anumită zi. Acest model permite adăugarea de noi tipuri de recurență fără a modifica codul existent (*Open/Closed Principle*).

Observer Pattern este implementat prin mecanismul de evenimente al Spring Framework. Clasa `EventCategoryChangedEvent` este un eveniment publicat de `CreateEventCategoryCommand` prin `ApplicationEventPublisher` atunci când o categorie de eveniment este modificată.

Ascultătorul `DailyLoaderScheduler` reacționează prin adnotarea `@EventListener` și reîncarcă programul zilnic, asigurând desincronizarea componentelor (*Low Coupling*).

Chain of Responsibility este folosit de Spring Security pentru procesarea cererilor HTTP. Filtrul `JwtAuthenticationFilter` face parte din lanțul de securitate și este poziționat înaintea filtrului `UsernamePasswordAuthenticationFilter`. Acesta extrage token-ul JWT din antetul cererii, îl validează și, dacă este valid, autentifică utilizatorul în contextul de securitate; în caz contrar, cererea este transmisă mai departe în lanț.

Adapter Pattern este utilizat pentru integrarea entității `User` cu Spring Security. Clasa `User` implementează interfața `UserDetails` și definește metodele `getAuthorities()`, `isAccountNonExpired()` și `isEnabled()`, adaptând modelul intern al utilizatorului la sistemul de autentificare al framework-ului (*Protected Variations*).

Repository Pattern este implementat prin Spring Data JPA.

Interfețe precum `UserRepository`, `EventRepository` și `EventCategoryRepository` extind `JpaRepository` și oferă operații standard de acces la date, precum și metode personalizate prin adnotări `@Query`, abstractizând stratul de persistență (*Indirection*).

Facade Pattern se regăsește în serviciile aplicației, precum `AuthenticationService` și `EventCategoryConfigService`. Acestea expun o interfață simplificată pentru operații complexe, coordonând mai multe componente interne și ascunzând detalii de implementare (*Pure Fabrication*).

Singleton Pattern este asigurat implicit de containerul Spring prin adnotările `@Service` și `@Component`. Clasa `TodayScheduleCache` este un exemplu explicit — un cache volatil partajat la nivelul întregii aplicații, care stochează programul evenimentelor din ziua curentă.

DTO Pattern este folosit pentru transferul datelor între straturile aplicației. Clase precum `CreateEventCategoryRequest`, `EventCategoryGetDTO`, `AttendanceGetDTO` și `UserCreatedDTO` asigură că entitățile interne nu sunt expuse direct către exterior, protejând integritatea datelor și permițând validări separate (*Protected Variations*).

4.3 Diagrame de interactiune

Diagramele de interactiune sunt folosite pentru a descrie modul in care interactioneaza componentele intre ele intr-un caz de utilizare. Ele se impart in doua tipuri : de secventa si de comunicare. Diagramele de secventa arata ordinea transmiterii mesajelor ,fiind mai flexibila decat cealalta. In continuare urmeaza sa prezint diagramele pentru cateva cazuri de utilizare.

4.3.1 Creare Cont

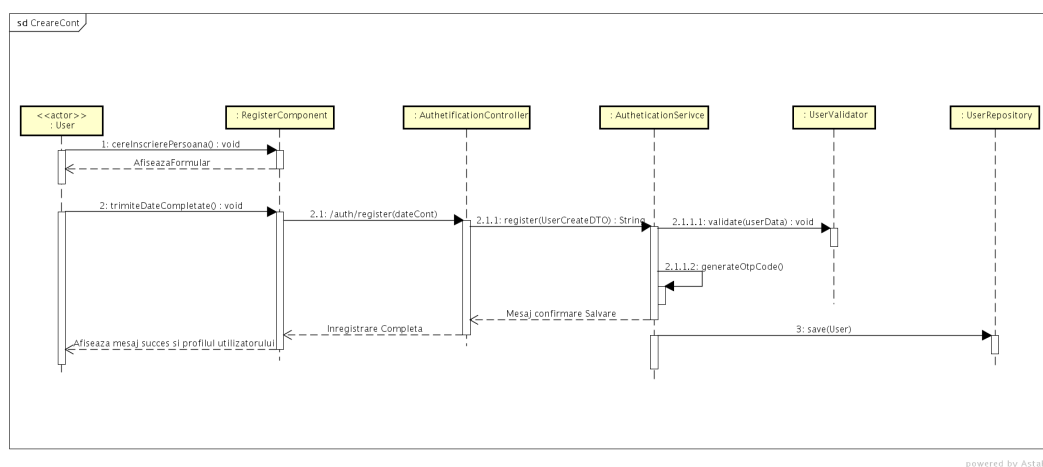


Figura 4.1: Diagrama de interacțiune — Creare cont

Conform diagramei de secventa, pentru implementarea acestui caz de utilizare am folosit clasa AuthenticationController pentru a separa fluxul de control in mai multe bucati mai usor de gestionat. Aceasta clasa se foloseste de AuthenticationService pentru a valida datele si pentru a crea obiectul de tip User. AuthenticationController contine metoda register(userCreateDTO) care apeleaza AuthenticationService.registerUser.

```
@PostMapping("/register")
public ResponseEntity<AuthenticationResponse> register(@RequestBody UserCreateDTO user) {
    return ResponseEntity.ok(service.register(user));
}
```

Figura 4.2: Implementare AuthenticationController

In AuthenticationService validam datele mai intai si dupa creem obiectul de tip User conform figurii 4.1

In imaginile de mai jos putem vedea interfata grafica. In prima utilizatorul introduce un

```

public AuthenticationResponse register(UserCreateDTO userCreateDTO) {
    userValidator.validate(userCreateDTO.getName(), userCreateDTO.getSurname(), userCreateDTO.getEmail());
    if (userCreateDTO.getPassword() == null || userCreateDTO.getPassword().length() < 6) {
        throw new IllegalArgumentException("Password must be at least 6 characters long");
    }

    User user = new User();
    user.setName(userCreateDTO.getName());
    user.setSurname(userCreateDTO.getSurname());
    user.setEmail(userCreateDTO.getEmail());
    user.setPassword(passwordEncoder.encode(userCreateDTO.getPassword()));
    user.setRole(UserRoles.USER);
    user.setActive(false);

    String otpCode = generateOtpCode();
    user.setMfaEnabled(true);
    user.setMfaCode(otpCode);
    user.setMfaExpiry(LocalDate.now().plusMinutes(5));
    repository.save(user);

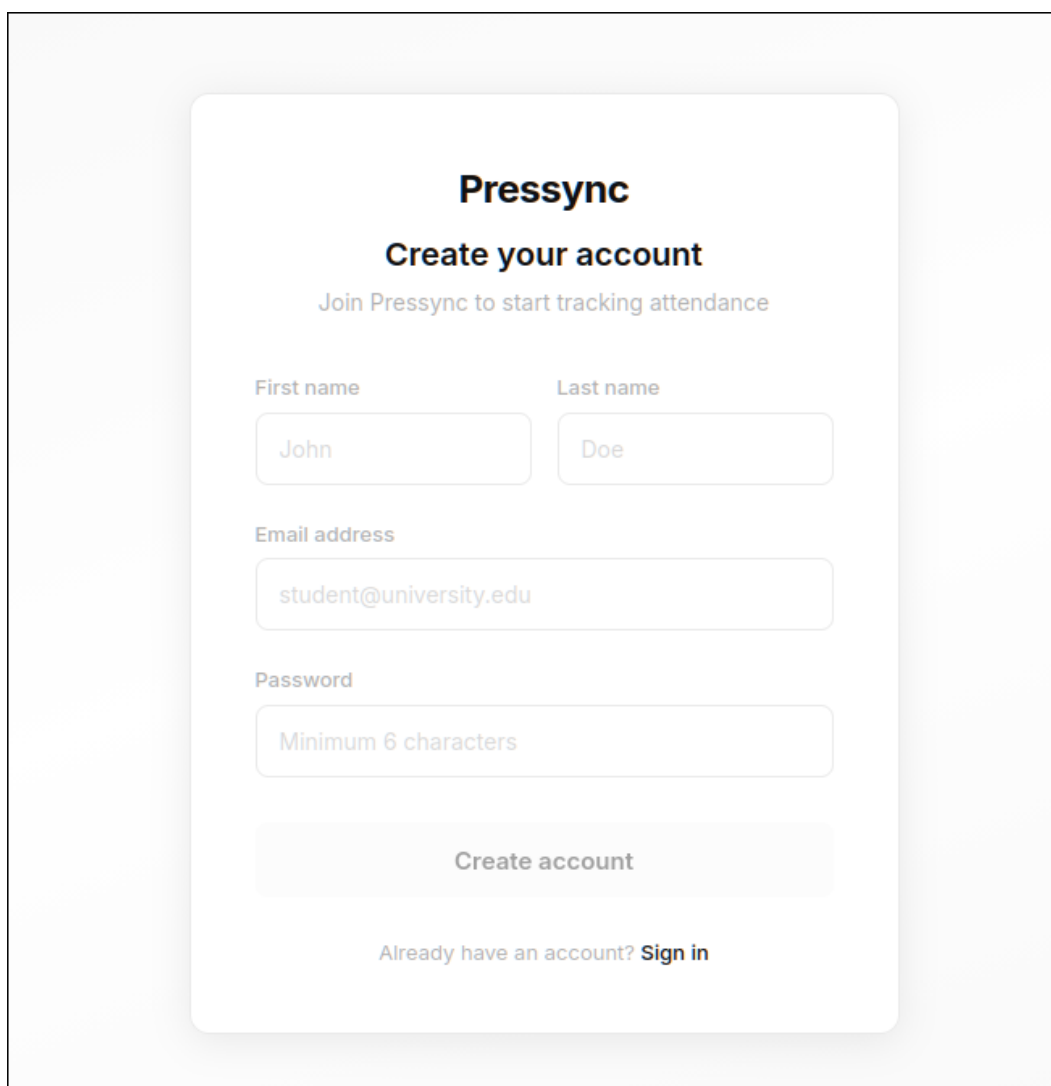
    emailService.sendOtpCode(user.getEmail(), otpCode);

    String maskedEmail = maskEmail(user.getEmail());
    return AuthenticationResponse.mfaRequired(maskedEmail);
}

```

Figura 4.3: Implementare AuthenticationService

nume sau prenume gresit , in a doua utilizatorul introuce un email gresit si in ultima o parola prea scurta



The image shows a registration form for 'Pressync'. The form is centered on a light gray background. It has a white background with rounded corners and a subtle drop shadow. The form contains the following elements: a title 'Pressync' in bold, a subtitle 'Create your account', a tagline 'Join Pressync to start tracking attendance', four input fields (First name, Last name, Email address, Password), a 'Create account' button, and a link 'Sign in' for existing users. The input fields contain placeholder text: 'John', 'Doe', 'student@university.edu', and 'Minimum 6 characters'.

Pressync

Create your account

Join Pressync to start tracking attendance

First name Last name

John Doe

Email address

student@university.edu

Password

Minimum 6 characters

Create account

Already have an account? [Sign in](#)

Figura 4.4: Interfața de înregistrare — formularul gol

The image shows a registration form with the following elements:

- First name:** A text input field containing "Vlad".
- Last name:** A text input field containing "Moraru".
- Email address:** A text input field containing "thorn". This field is highlighted with a red border, and a red error message "Please enter a valid email address." is displayed below it.
- Password:** A password input field with 12 black dots, highlighted with a yellow background.
- Create account:** A light gray button.
- Sign in:** A link labeled "Sign in" in blue text, preceded by the text "Already have an account?" in gray.

Figura 4.6: Interfața de înregistrare — eroare email invalid

The image shows a registration form with the following fields and elements:

- First name:** A text input field containing "Vlad".
- Last name:** A text input field containing "Moraru".
- Email address:** A text input field containing "thorn@example.com", highlighted with a yellow background.
- Password:** A password input field with two black dots, outlined in red. Below it, a red error message reads: "Password must be at least 6 characters."
- Create account:** A light gray button.
- Sign in:** A link labeled "Sign in" preceded by the text "Already have an account?".

Figura 4.7: Interfața de înregistrare — eroare parolă prea scurtă

4.3.2 Creare Evenimente

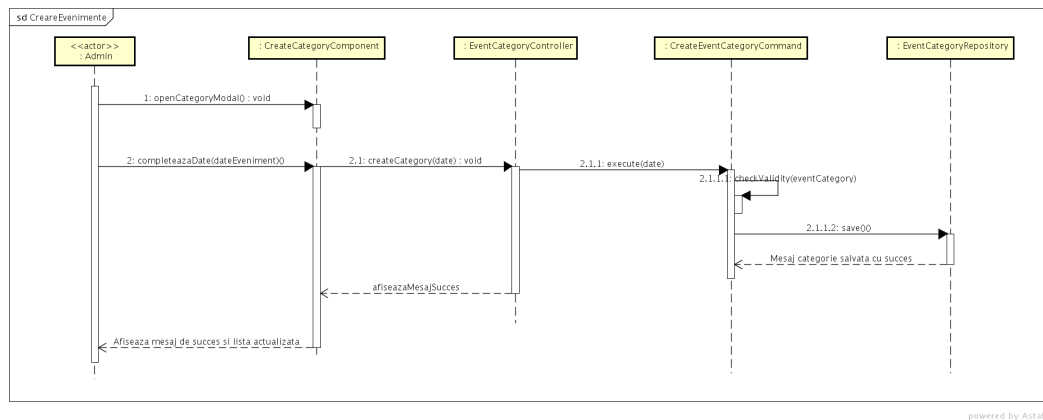


Figura 4.8: Diagrama de interacțiune — Creare evenimente

Pentru acest caz de utilizare am implementat componenta frontend `CreateCategoryComponent` care apelează după completarea cu succes a formularului endpointul de tip `POST event-Category/data`

```
@PostMapping("/{create}") @ thorn
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<String> addEventCategory(@RequestBody CreateEventCategoryRequest eventCategoryRequest){
    return createEventCategoryCommand.execute(eventCategoryRequest);
}
```

Figura 4.9: Implementare `EventCategoryController`

Controllerul la randul sau apelează metoda `execute` a clasei `CreateEventCategoryCommand`

```

public ResponseEntity<String> execute(CreateEventCategoryRequest request) {
    EventCategory entity = new EventCategory();
    entity.setName(request.name());
    entity.setStartingTime(request.startingTime());
    entity.setEndTime(request.endTime());
    entity.setAttendanceTimeStart(request.attendanceTimeStart());
    entity.setAttendanceDuration(request.attendanceDuration());
    entity.setRepeatable(request.repeatable());
    if (Boolean.TRUE.equals(request.repeatable())) {
        EventCategoryConfig config;
        if (request.configId() != null) {
            eventCategoryConfigService.enforceSingleCategoryPerConfig(request.configId(), categoryIdToIgnore: null);
            config = eventCategoryConfigRepository.findById(request.configId())
                .orElseThrow(() -> new IllegalArgumentException("Config not found"));
            eventCategoryConfigService.validateConfig(config);
        } else {
            config = new EventCategoryConfig();
            config.setRepeatableType(request.repeatableType());
            config.setRepeatsOnSpecificDay(request.repeatsOnSpecificDay());
            config.setBaseDate(request.baseDate() != null ? request.baseDate() : LocalDate.now());
            eventCategoryConfigService.validateConfig(config);
            config = eventCategoryConfigRepository.save(config);
        }
        entity.setCategoryConfig(config);
    } else {
        entity.setSpecificDate(request.baseDate() != null ? request.baseDate() : LocalDate.now());
    }

    checkValidity(entity);
    if (isCollidingWithSomething(entity)){
        throw new IllegalArgumentException("Colliding with something");
    };

    eventCategoryRepository.save(entity);
    applicationEventPublisher.publishEvent(new EventCategoryChangedEvent(entity));
    return ResponseEntity.ok( body: "{}");
}

```

Figura 4.10: Implementare CreateEventCategoryCommand

Aceasta clasa creaza obiectul de tip EventCategory si valideaza atat configul , adica partea ce tine de programarea evenimentelor urmatoare , sa nu se ciocneasca cu alte evenimente, atat si ca datele introduse sa fie valide. In continuare prezint interfata grafica pentru acest caz de utilizare impreuna cu toate erorile pe care aceasta le poate afisa.



Figura 4.11: Interfața de creare categorii — butonul de creare

Figura 4.12: Interfața de creare categorii — panoul de creare

Create New Category

×

Invalid attendance start time

Event Schedule

Category Name *

Curs Blockchain

Start Time *

12 : 00

End Time *

14 : 00

12:00

14:00

Opens 11:01 59 min before

☐ Recurring event

Attendance Window

Opens at *

11 : 01

Duration *

5 min

20 min

179 min

Figura 4.13: Interfața de creare categorii — eroare 1

Create New Category

×

Invalid attendance start time

Event Schedule

Category Name *

Curs Blockchain

Start Time *

12 : 00

End Time *

11 : 00

12:00

Opens 11:00 60 min before

11:00

Recurring event

Attendance Window

Opens at *

11 : 00

Attendance must open before the event ends.

Figura 4.14: Interfața de creare categorii — eroare 2

Create New Category

×

Invalid start time or end time

📅

Event Schedule

Category Name *

Curs Blockchain

Start Time *

12 : 00

End Time *

11 : 00

12:00

11:00

Opens 11:00 60 min before

☐

Recurring event

🕒

Attendance Window

Opens at *

11 : 00

⚠

Attendance must open before the event ends.

Figura 4.15: Interfața de creare categorii — eroare 3

4.3.3 Înregistrare Prezența

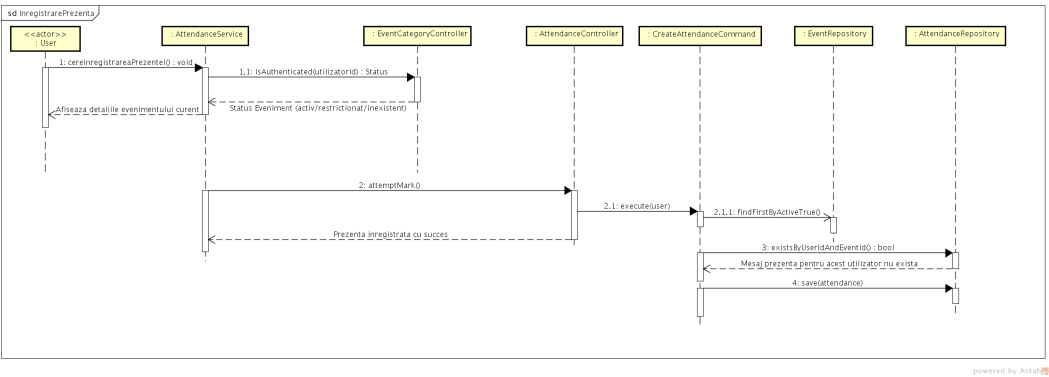


Figura 4.16: Diagrama de interacțiune — Înregistrare prezență

//placeholders code

4.3.4 Vizualizare prezenta cu statistici

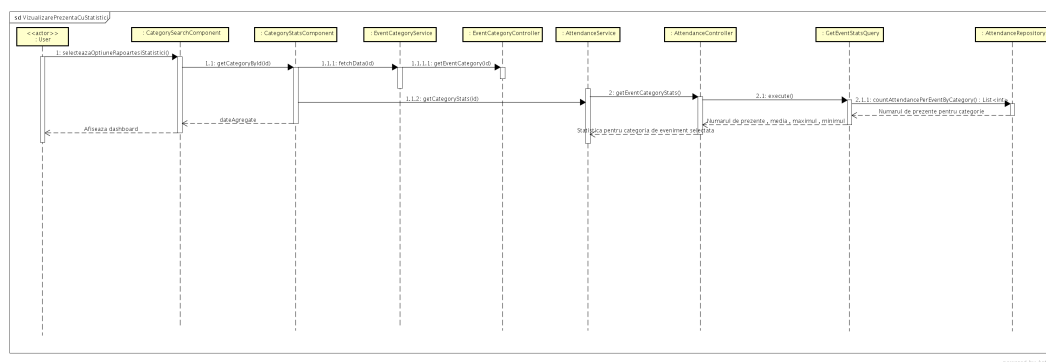


Figura 4.17: Diagrama de interacțiune — Vizualizare prezentă cu statistici